

Project Report

Course: DL & GenAI | **Roll No:** 24DS3000019 | **Competition:** Smart MCQ Solver Challenge

1. Abstract / Executive Summary

This report presents an NLP pipeline built for Kaggle's **Smart MCQ Solver Challenge**, where the task is to read a question prompt with five answer choices (A–E) and output a ranked top-3 prediction, scored via **Mean Average Precision @ 3 (mAP@3)**.

The problem was tackled in three phases spanning the evolution of NLP techniques: **(1)** a non-parametric **TF-IDF + Cosine Similarity** baseline, **(2)** a custom PyTorch network (**SimpleMCQNet**) that learns weights over concatenated TF-IDF vectors, and **(3)** a fine-tuned Transformer (**DistilBERT**) using Hugging Face's **Trainer** API with subword tokenization and self-attention.

Results showed a clear hierarchy: the TF-IDF baseline scored **0.2576** mAP@3, the PyTorch scratch model improved this to **0.3603**, and fine-tuned DistilBERT achieved **0.7472** — nearly double the scratch model and almost 3x the baseline.

Key takeaway: word-level tokenization cannot capture semantic relationships or word order. A from-scratch neural network improves on raw similarity by learning weighted alignments, but the largest gains come from **Transfer Learning** — pre-trained attention mechanisms generalize far better with limited data (2,000 rows) than any model trained from scratch.

2. Introduction

Problem Statement: The competition requires a model that reads a **prompt** and five options (A–E) and predicts the top three most probable answers, evaluated via mAP@3, which rewards placing the correct answer higher in the ranking.

Objectives: Beyond leaderboard performance, this project aimed to (1) experience the architectural evolution of NLP — from TF-IDF vectorization, to custom PyTorch networks, to pre-trained Transformers; (2) build hands-on proficiency with **PyTorch** and **Hugging Face** (**transformers**, **datasets**); and (3) apply MLOps practices using **Weights & Biases (W&B)** for experiment tracking.

Report Structure: Section 3 covers the dataset and preprocessing; Section 4 covers tokenization strategy; Section 5 details the three modeling phases; Section 6 compares model performance; Section 7 concludes with key learnings and future work; Section 8 lists references.

3. Dataset & Preprocessing

Dataset: The official Kaggle dataset includes a **training set** of 2,000 records (**id**, **prompt**, A–E, **answer**) and a **test set** of 500 records (same columns, minus **answer**).

EDA Highlights:

- **Data Integrity:** No missing values across either set — no imputation required.

- **Class Balance:** Answer distribution was A: 369, B: 490, C: 459, D: 358, E: 324 — mildly skewed toward B/C but no severe imbalance. A majority-class baseline would only reach ~24.5% accuracy, confirming mAP@3 reflects genuine model learning.
- **Text Length:** Prompts average ~18 words (max 51); options average ~26 words (max ~85). Train/test length distributions matched closely, and these max lengths directly informed the `max_length` hyperparameter for transformer tokenization.

Preprocessing: For the TF-IDF-based models (Phases 1–2), text was lowercased and stripped of punctuation to control vocabulary size. Transformers (Phase 3) retained original punctuation, since subword tokenizers use it to infer grammatical structure. No synthetic data augmentation was applied — the full 2,000-row set was used as-is to keep a controlled, comparable environment across all three models.

4. Tokenization Strategy

The final model used Hugging Face's `AutoTokenizer` for **DistilBERT**, which applies **WordPiece** subword tokenization. This was chosen over word-level tokenization for two reasons:

1. **OOV Handling:** Unknown or complex words are broken into subword units (e.g., "tokenization" → "token" + "##ization"), so no information is discarded.
2. **Contextual Preservation:** Retaining punctuation and subword structure lets the self-attention mechanism understand relational context — e.g., distinguishing a sentence's subject from its object — which word-level Bag-of-Words methods cannot do.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Transformer (DistilBERT)

Architecture: `distilbert-base-uncased`, fine-tuned via Hugging Face's `AutoModelForMultipleChoice`. DistilBERT (Sanh et al., 2019) distills BERT-base's 12 layers down to **6 Transformer layers** (768 hidden dim, 12 attention heads, ~66M parameters), retaining ~97% of BERT's language understanding at 60% faster inference and 40% less memory.

For each question, five (prompt, option) pairs are formatted as `[CLS] Prompt [SEP] Option [SEP]`, and the `[CLS]` representation for each option is passed through a shared linear classifier, producing 5 logits scored with `CrossEntropyLoss`.

Fine-Tuning Setup: Optimizer **AdamW** (weight decay 0.01) with a learning rate of 2×10^{-5} , linear warmup (10% of steps) plus decay, **no frozen layers** (full fine-tuning across all 6 blocks), 3–5 epochs, batch size 8, with `fp16=True` mixed precision on Kaggle's dual T4 GPUs.

5.2 Model 2: Custom PyTorch Network (SimpleMCQNet)

Architecture: A from-scratch feed-forward network that learns relationships between prompt and option TF-IDF vectors, rather than relying on raw geometric similarity. Each prompt and option is vectorized to 1,000 dimensions and concatenated into a 2,000-dim input, passed through a single dense layer (`nn.Linear(2000, 1)`). All 5 options per question are processed together and reshaped to `(Batch, 5)`, scored with `CrossEntropyLoss`.

[Prompt (1000)] + [Option (1000)] → Concat (2000) → Linear Layer → Score

Key Points: Concatenation (rather than subtraction) lets the network independently learn which prompt words align with which option words. Capping vocabulary at 1,000 words and using a single linear layer kept the model lightweight and prevented overfitting on only 2,000 training rows.

5.3 Model 3: Heuristic Baseline (TF-IDF + Cosine Similarity)

Architecture: A zero-training statistical baseline using Scikit-Learn. A `TfidfVectorizer` (max 5,000 features) was fit across all prompts and options; each prompt's vector was compared to its five option vectors via **Cosine Similarity**, and options were ranked by similarity score to produce the top-3.

Key Points: This establishes a mathematical floor — any learned model must beat it to prove architectural value. It also exposed the core weakness of Bag-of-Words: reliance on exact keyword overlap with no grasp of context, synonyms, or negation.

6. Performance & Comparative Analysis

Metric: All models were scored on **mAP@3**, the official competition metric (Rank 1 = 1.0 pts, Rank 2 = 0.5 pts, Rank 3 = 0.33 pts, not in top 3 = 0 pts). CrossEntropyLoss was tracked during training via **Weights & Biases**. The scratch model's loss stabilized quickly (~1.6098 → ~1.6092 over 5 epochs), while DistilBERT showed a much steeper, more effective convergence.

Comparative Table:

Model Architecture	Tokenization / Features	Parameters	mAP@3 Score
Model 3: Baseline	TF-IDF (Word-Level)	0 (Heuristic)	0.2576
Model 2: PyTorch Scratch	TF-IDF (Word-Level)	~2,000	0.3603
Model 1: DistilBERT	WordPiece (Subword)	~66 Million	0.7472

Analysis: The baseline barely exceeded random guessing (~0.25 vs. 0.20), relying purely on keyword overlap. The PyTorch scratch model improved this by ~40% (to 0.360) by learning weights over those same keywords. DistilBERT provided a massive leap to 0.747, confirming that contextual self-attention and subword tokenization are decisively superior for multiple-choice reasoning. The best Kaggle submission (Version 18) used the fine-tuned Transformer, scoring **0.74729** on the leaderboard.

7. Conclusion & Future Work

Key Learnings: TF-IDF is fast and useful for baselines but treats language as isolated tokens. Building the PyTorch scratch model clarified the mechanics of tensor manipulation, dataset formatting, and gradient updates. Fine-tuning DistilBERT demonstrated the power of Transfer

Learning — pre-trained semantic weights drastically cut the data needed for strong performance on complex reasoning tasks.

Challenges Faced: Reformatting the tabular data so the tokenizer paired each prompt with all 5 options (`[batch, 5, seq_len]` tensor) required careful reshaping for the multiple-choice head. Fine-tuning a 66M-parameter model also demanded careful memory management — mixed precision (`fp16=True`), dual T4 GPUs, and tight batch-size limits to avoid CUDA OOM errors.

Future Work:

- **RAG:** Add a FAISS-based vector retrieval layer for external context on technical questions.
- **Larger Architectures:** Try DeBERTa-v3, which generally outperforms standard BERT on MCQ tasks via disentangled attention.
- **Weighted Ensembling:** Combine logits from multiple architectures (e.g., DistilBERT + RoBERTa) for a stronger final ranking.

8. References

1. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv:1910.01108.
2. Hugging Face Documentation. *Transformers and Datasets Libraries*. huggingface.co/docs.
3. PyTorch Documentation. *Tensors and Neural Networks (torch.nn)*. pytorch.org/docs.
4. Weights & Biases (wandb). *Experiment Tracking*. wandb.ai.
5. Kaggle. *Smart MCQ Solver Challenge Dataset*. kaggle.com/competitions/smart-mcq-solver-challenge.